- Each hospital is a Docker container to prove isolation
- The 2 most common algorithms for combining the weights in federated learning
    - FedAvg - The central server accepts changed model weights every round and updates the shared weights proportionally to each “hospitals” dataset size
        - Read more here https://arxiv.org/abs/1602.05629
        - A round can be several epochs. We can tune this parameter.
    - FedProx - An extension of FedAvg to handle non-independent data and difference in device capabilities
        - Given how diverse our data is (some hospitals have 60% tumor rate while others closer to 5%), this may be worth using
        - Read more here https://arxiv.org/abs/1812.06127

The FedAvg loop:

1. The server has a set of clients (hospitals)
2. The server gives them the current global weights
3. Every client trains locally however it wants
4. The clients send new weights and number of samples
5. Server performs FedAvg
6. Server evals this, then start loop again

The slightly different FedProx loop:

1. The server has a set of clients (hospitals)
2. The server gives them the current global weights
3. Every client trains locally, but gets penalized for moving too far from the global weights
4. The clients send new weights and number of samples

5. Server combines the weights proportionally to each hospital's dataset size (same as FedAvg)
6. Server evals this, then start loop again

Networking

We should probably use gRPC to send data between clients/server since weights are binaries and not json. It is also more suitable for large amounts of data but the weights for our models will likely be tiny so this isn't a factor. But still good to be scalable. We can also hold persistent 2 way connections for training loop instead of making new connection everytime

Library or no library

There are federated learning python libraries (ex. Flower https://github.com/flwrlabs/flower) that handle the coordination logic for us. They include implementations of FedAvg and FedProx, the 6 step loop above, gRPC networking setup, Docker images. Crucially, these libraries would not be used for the individual model/training code. Only the coordination process.

We need to make a decision on whether we will implement the federated coordination from scratch or use one of these libraries. On one hand, the aim is for us to do modeling from scratch to learn more. On the other, much of this will be networking/ops/swe work rather than implementing models. For that reason, I am recommending we use the library but am open to feedback from anyone.